DOCUMENTO DE ARQUITETURA E AUDITORIA

Plataforma Escalável de Visão Computacional para Detecção e Catalogação de Objetos em Vídeo

Escopo inicial do piloto • evolução arquitetural para produção • auditoria comparativa entre perfis operacionais, engines de inferência e ambientes de execução

Objetivo do documento	Consolidar o planejamento do projeto, detalhando visão, requisitos, arquitetura, algoritmos, auditoria e roadmap de implementação.
Stack principal	C no núcleo crítico; FFmpeg para vídeo; engines plugáveis de inferência; PostgreSQL, fila de jobs e storage desacoplado.
Foco operacional	Baixa latência, escalabilidade horizontal, comparabilidade experimental e rastreabilidade total de execução.
Uso esperado	Base de alinhamento técnico, planejamento de sprints, desenho de MVP e referência para evolução até produção.

Versão 1.0 • Documento consolidado para planejamento técnico

1. Resumo Executivo

O projeto parte de um piloto básico de visão computacional com entrada em vídeo, detecção de objetos e catalogação estruturada, mas evolui deliberadamente para uma plataforma de processamento distribuído orientada a jobs. O núcleo crítico permanece em C para reduzir overhead, controlar memória, melhorar previsibilidade temporal e sustentar latência baixa em cenários de escala.

A arquitetura alvo separa ingestão, orquestração, processamento, persistência, consulta e auditoria. Essa separação evita acoplamento entre recepção de demanda e execução intensiva, permite múltiplos workers, suporta filas, habilita pools especializados de execução e cria trilha de auditoria reproduzível para comparar engines, perfis operacionais, versões de modelo e ambientes de hardware.

O documento foi escrito para servir como base de engenharia. Ele combina visão funcional, desenho de arquitetura, modelo de dados, algoritmização do pipeline, regras de auditoria comparativa, matriz de experimentos, indicadores operacionais, backlog evolutivo e roadmap de implementação.

Dimensão	Decisão de projeto	Intenção prática
Core de processamento	C	Baixa latência, controle de memória e previsibilidade operacional
Entrada de mídia	Vídeo batch inicialmente	Reduz risco do MVP e simplifica auditoria
Inferência	Engine plugável	Permite benchmark entre ONNX Runtime, TensorRT, OpenVINO etc.
Escala	Workers distribuídos + fila	Aumento horizontal de throughput com isolamento operacional
Auditoria	Snapshots completos + comparação controlada	Comparar perfis, engines, hardware e configuração com justiça experimental

2. Premissas, objetivos e princípios

2.1 Premissas estruturantes

- O sistema receberá vídeos piloto e, depois, lotes de vídeos ou referências para assets em storage.
- A solução deve nascer com separação entre experimentação técnica e execução operacional, para que benchmarking e produção compartilhem o mesmo núcleo de processamento.
- Treinamento de CNN não faz parte do núcleo em C; o modelo é exportado e consumido por uma engine de inferência compatível.
- Comparações entre engines e perfis só são válidas quando vídeo, modelo, thresholds, política de sampling e ambiente de hardware forem controlados ou explicitamente versionados.
- Toda execução relevante deve deixar rastros suficientes para ser reconstituída, auditada e comparada posteriormente.

2.2 Objetivos de negócio e engenharia

1. Processar vídeos e detectar objetos com desempenho consistente.

2. Catalogar objetos de modo útil para análise posterior, evitando contagens infladas por repetição frame a frame.
3. Suportar crescimento por adição de workers, sem precisar redesenhar o núcleo do pipeline.
4. Permitir comparação objetiva entre engines, modelos, perfis operacionais e ambientes.
5. Gerar informações operacionais que sustentem decisão técnica: qual configuração entrega melhor latência, throughput, custo e estabilidade.

2.3 Princípios arquiteturais

Princípio	Descrição operacional
Desacoplamento	Ingestão, fila, worker e persistência não devem depender da mesma unidade de execução.
Observabilidade nativa	Métricas, logs e auditoria não são anexos; fazem parte do desenho base.
Comparabilidade	Toda mudança de engine, modelo, perfil ou hardware precisa ser mensurável e repetível.
Substituibilidade	A engine de inferência deve ser trocada sem reescrever o pipeline funcional.
Evolução incremental	O projeto avança do MVP técnico para operação distribuída sem abandono do core já validado.

3. Escopo do projeto

3.1 Escopo inicial do piloto

- Receber um vídeo piloto em formato suportado.
- Decodificar frames do vídeo.
- Aplicar um modelo CNN de detecção de objetos via engine de inferência.
- Registrar as detecções por frame, timestamp, classe, confiança e bounding box.
- Consolidar um catálogo resumido dos objetos identificados.
- Exportar os resultados em formato estruturado, como CSV e JSON.

3.2 Ampliação de escopo para arquitetura operacional

- Submissão assíncrona de jobs com identificador único.
- Fila de jobs e dispatcher para distribuição de carga.
- Workers independentes com pipeline modular.
- Persistência de resultados e métricas por execução.
- Camada explícita de auditoria, benchmark e score operacional.
- Comparação controlada entre perfis operacionais, engines de inferência, modelos e ambientes.

3.3 Fora de escopo da primeira onda

- Treinamento de modelos no núcleo em C.
- Reidentificação avançada ou tracking de alta complexidade.

- Streaming online de baixa latência extrema em múltiplas câmeras simultâneas.
- Interface gráfica completa de supervisão em tempo real.
- Processamento multimodal, segmentação semântica e pipelines de anotação humana.

4. Requisitos consolidados

4.1 Requisitos funcionais

ID	Requisito
RF01	O sistema deve registrar um job para cada submissão de vídeo.
RF02	O sistema deve associar ao job o perfil operacional, a engine de inferência, a versão do modelo e o ambiente de execução.
RF03	O worker deve decodificar frames e produzir amostras conforme política configurada.
RF04	O pré-processamento deve converter cada frame para o formato esperado pelo modelo.
RF05	A engine de inferência deve processar tensores e retornar saídas padronizadas.
RF06	O pós-processamento deve aplicar thresholds, NMS e normalização de classes.
RF07	O sistema deve associar detecções entre frames quando a política de tracking estiver habilitada.
RF08	O sistema deve construir um catálogo agregando ocorrências por classe e por objeto observado.
RF09	O sistema deve persistir resultados detalhados, métricas por estágio e snapshots de parâmetros.
RF10	O sistema deve comparar execuções e gerar relatórios auditáveis.

4.2 Requisitos não funcionais

ID	Requisito	Interpretação técnica
RNF01	Baixa latência	Evitar overhead desnecessário, reuso de buffers e desenho de pipeline previsível.
RNF02	Escalabilidade horizontal	Aumento de capacidade por multiplicação de workers.
RNF03	Rastreabilidade	Toda execução relevante precisa ser reconstituível.
RNF04	Substituibilidade de engine	O backend de inferência não pode contaminar a lógica de domínio.
RNF05	Comparabilidade	Resultados de benchmark devem seguir matriz controlada de experimentação.
RNF06	Resiliência	Jobs precisam de estados claros, retry controlado e isolamento de falhas.

5. Arquitetura alvo da plataforma

A arquitetura de produção é orientada a jobs e organizada em camadas. O objetivo não é apenas processar vídeos, mas sustentar operação, comparação experimental e evolução contínua.

```
[Cliente / API / Upload]
          |
          v
 [Serviço de Ingestão]
          |
          v
 [Fila / Broker de Jobs]
          |
          v
 [Agendador / Dispatcher]
          |
   -------------------------
   |           |           |
   v           v           v
[Worker]    [Worker]    [Worker]
   |           |           |
   v           v           v
[Decode -> Sample -> Preprocess -> Infer -> Postprocess -> Track -> Catalog]
          |
          v
 [Persistência + Auditoria + Métricas]
          |
          v
 [Consulta / Export / Relatórios]
```

5.1 Serviço de ingestão

A ingestão é a fronteira do sistema. Ela recebe o vídeo ou a referência do asset, valida o formato, registra o job, aplica políticas iniciais e devolve um identificador de rastreamento. Esse serviço não executa inferência e não deve concentrar carga computacional pesada.

- Validar metadados mínimos: tamanho, codec, duração, origem, integridade e permissões de acesso.
- Persistir o registro inicial do job com status RECEIVED ou QUEUED.
- Resolver o perfil operacional padrão quando o usuário não informar explicitamente.
- Calcular assinatura da configuração submetida antes do despacho.
- Publicar o job na fila apropriada, considerando prioridade e pool de execução.

5.2 Broker de jobs

O broker desacopla chegada de trabalho e processamento real. Ele absorve picos de entrada, distribui jobs para workers e permite políticas de retry, fila prioritária, dead-letter e backpressure.

5.3 Scheduler / dispatcher

O dispatcher decide qual worker ou pool deve atender determinado job. A decisão pode considerar perfil operacional, urgência, engine necessária, compatibilidade de hardware e ocupação do cluster.

Entrada de decisão	Exemplo de uso no dispatcher
Perfil low latency	Enviar para pool com fila curta, batch size 1 e preferência por hardware mais rápido.
Perfil throughput	Agrupar em pool que aceita maior backlog e aproveita melhor batching.
Engine TensorRT	Despachar somente para nós com GPU e runtime compatível.
Prioridade alta	Ultrapassar jobs comuns, preservando cotas e evitando starvation.

6. Arquitetura interna do worker

O worker é a unidade de processamento crítico. Ele deve ser modularizado para facilitar profiling, reuso e substituição de componentes. A organização recomendada é pipeline-oriented, com estágios definidos e métricas por etapa.

```
Job Loader
   -> Video Decoder
   -> Frame Sampler
   -> Preprocessor
   -> Inference Backend
   -> Postprocessor
   -> Tracker / Deduplicator
   -> Catalog Builder
   -> Metrics Collector
   -> Result Publisher
```

Módulo	Responsabilidade
Job Loader	Carrega o contexto do job, resolve paths, parâmetros e recursos de runtime.
Video Decoder	Abre o vídeo via FFmpeg, decodifica quadros e converte para formato intermediário estável.
Frame Sampler	Seleciona todos os frames ou uma amostragem configurada por política temporal ou estrutural.
Preprocessor	Aplica resize, letterbox, normalização, reorder de canais e montagem do tensor de entrada.
Inference Backend	Encapsula a engine escolhida e executa inferência com interface unificada.
Postprocessor	Interpreta a saída bruta, aplica threshold, NMS e mapeia classes reconhecidas.
Tracker	Associa detecções ao longo do tempo e reduz inflação de contagem.
Catalog Builder	Consolida resultados por classe, objeto, janela temporal e resumo final do job.
Metrics Collector	Mede latência, memória, taxa de erro, throughput e percentis por estágio.
Result Publisher	Persiste resultados, publica eventos finais e encerra a trilha auditável.

6.1 Estratégia de paralelismo

A primeira fonte de escala é o paralelismo entre jobs: vários vídeos processados em workers distintos. Em seguida, dentro de um job, o pipeline pode ser concorrente por estágio, com buffers intermediários e threads dedicadas. O paralelismo interno deve ser introduzido de forma incremental para não esconder gargalos reais sob complexidade prematura.

- Nível 1: entre jobs — mecanismo principal de escalabilidade horizontal.
- Nível 2: entre estágios — decode, preprocess, inferência e pós-processamento desacoplados por filas locais.
- Nível 3: dentro da engine — uso do paralelismo e da aceleração oferecidos pelo runtime escolhido.

6.2 Estratégia de memória e buffers

- Evitar malloc/free por frame; preferir pools de objetos e buffers reutilizáveis.
- Separar pools para frame decodificado, tensor de entrada, saída de inferência e estruturas de detecção.
- Usar buffers contíguos e alinhados quando exigido pela engine ou pela estratégia SIMD.
- Minimizar cópias entre estágios; cada conversão desnecessária encarece a latência total do job.
- Estabelecer limites de fila interna por estágio para evitar crescimento descontrolado de memória.

7. Contrato de abstração da inferência

A engine de inferência deve ficar atrás de uma interface uniforme. O pipeline não pode conhecer APIs específicas de ONNX Runtime, TensorRT ou OpenVINO. Essa decisão é essencial para benchmark justo, substituição segura e manutenção de longo prazo.

```
typedef struct InferenceBackend InferenceBackend;

typedef struct {
    int (*init)(InferenceBackend *backend, const char *model_path, void *config);
    int (*infer)(InferenceBackend *backend, const float *input, float *output);
    int (*get_output_shape)(InferenceBackend *backend, int *shape, int max_dims);
    void (*destroy)(InferenceBackend *backend);
} InferenceBackendOps;
```

- init: inicializa a engine com caminho de modelo e configuração operacional.
- infer: recebe o tensor padronizado e produz saída bruta conforme contrato interno.
- get_output_shape: permite ao pipeline adaptar pós-processamento e buffers de saída.
- destroy: libera recursos, sessões, contexto de GPU e memória específica do backend.

Engine	Quando faz sentido	Aspecto que a auditoria deve comparar
ONNX Runtime	Portabilidade e baseline neutra	Latência média, estabilidade e custo de integração
TensorRT	Ambiente NVIDIA com foco em desempenho	Ganho de throughput/latência versus custo de dependência
OpenVINO	Ambiente Intel e otimização em	Eficiência e comportamento em hardware

Engine	Quando faz sentido	Aspecto que a auditoria deve comparar
	CPU/VPU	específico

8. Fluxo algorítmico do processamento

8.1 Algoritmo macro do job

```
processar_job(job):
    carregar_contexto(job)
    registrar_inicio_auditoria(job)
    inicializar_buffers(job)
    inicializar_engine(job)

    para cada frame_decodificado do video:
        if sampler_deve_processar(frame_decodificado):
            tensor = preprocessar(frame_decodificado)
            saida_bruta = inferir(tensor)
            deteccoes = posprocessar(saida_bruta)
            deteccoes_associadas = rastrear(deteccoes)
            catalogar(deteccoes_associadas)
            medir_metricas_por_estagio()

    persistir_resultado_detalhado(job)
    persistir_resumo_catalogado(job)
    consolidar_metricas(job)
    registrar_fim_auditoria(job)
    publicar_status(COMPLETED)
```

8.2 Decodificação e sampling

A decodificação precisa produzir um fluxo consistente de frames. O sampling define a política de passagem para inferência e impacta diretamente custo, latência e fidelidade do catálogo. Por isso, ele deve ser parte explícita do perfil operacional e da trilha de auditoria.

- Sampling completo: processa todos os frames elegíveis e maximiza granularidade temporal.
- Sampling por intervalo fixo: processa um frame a cada N frames ou a cada X milissegundos.
- Sampling adaptativo: altera a taxa de amostragem em função de movimento, mudança de cena ou estado do job.

8.3 Pré-processamento

O pré-processamento deve ser determinístico. Pequenas diferenças na preparação do tensor podem inviabilizar a comparação entre engines. O pipeline precisa registrar resolução final, política de resize, preenchimento, normalização, ordem dos canais e precisão numérica efetivamente utilizada.

6. Receber frame no formato intermediário padrão do worker.
7. Aplicar resize ou letterbox respeitando a dimensão esperada pelo modelo.
8. Normalizar os pixels conforme contrato do modelo exportado.
9. Reordenar o layout dos canais quando necessário.

10. Escrever o resultado no buffer de tensor associado ao job.

8.4 Pós-processamento e tracking

O pós-processamento transforma a saída bruta da engine em detecções utilizáveis. Depois disso, o tracking básico reduz inflação de contagem e aproxima o catálogo da noção de “objeto observado” e não apenas “detecção por frame”.

```
posprocessar(saida_bruta):
    caixas = decodificar_caixas(saida_bruta)
    caixas = filtrar_por_confianca(caixas, threshold)
    caixas = aplicar_nms(caixas, iou_threshold)
    caixas = mapear_classes(caixas)
    retornar caixas

rastrear(deteccoes_atuais, tracks_anteriores):
    associacoes = casar_por_iou(deteccoes_atuais, tracks_anteriores)
    atualizar_tracks_existentes(associacoes)
    criar_novos_tracks_para_nao_associados()
    encerrar_tracks_inativos()
    retornar deteccoes_com_track_id
```

9. Perfis operacionais

Perfis operacionais são conjuntos pré-definidos de parâmetros que alinham a execução a uma meta prioritária. Eles não são apenas “presets”, mas hipóteses de operação que precisam ser auditadas e comparadas sistematicamente.

Perfil	Meta principal	Configuração típica
Low Latency	Menor tempo de resposta	Batch size 1, fila curta, modelo leve, sampling conservador, preferência por hardware rápido
High Throughput	Maior volume por unidade de tempo	Batching, backlog maior, melhor ocupação de GPU/CPU, tolerância a latência maior
Cost Efficient	Menor custo computacional	Sampling mais agressivo, resolução reduzida, modelo menor e limites de memória mais rígidos
Balanced	Compromisso entre desempenho e qualidade	Parâmetros médios para operação geral

9.1 Estrutura de um perfil

- Resolução de entrada e política de resize.
- Sampling temporal e tamanho máximo de fila.
- Thresholds de confiança e IoU.
- Batch size, engine preferencial e tipo de hardware.
- Política de tracking, timeout e retry.
- Limites de memória e regras de fallback operacional.

10. Modelo de dados da plataforma

A persistência deve separar metadados operacionais, detecções detalhadas, resumos catalogados e trilhas de auditoria. Isso reduz acoplamento entre consulta transacional, análise comparativa e operação do worker.

Entidade	Responsabilidade	Campos mínimos
video_job	Representa a unidade operacional submetida	job_id, source_uri, status, priority, profile_id, engine_id, model_version_id
processing_profile	Congela o conjunto de parâmetros do perfil	profile_id, name, sampling_policy, input_resolution, thresholds, batch_size
inference_engine	Versiona o backend executado	engine_id, name, version, backend_type, device_type, precision_mode
runtime_environment	Descreve o host de execução	runtime_env_id, cpu_model, gpu_model, memory_gb, os_version, driver_version
frame_detection	Armazena as detecções detalhadas por frame	job_id, frame_number, timestamp_ms, class_id, confidence, bbox, track_id
catalog_summary	Guarda o resumo catalogado do job	job_id, class_id, total_detections, total_unique_tracks, first_seen_ms, last_seen_ms
processing_metrics	Persistência das medições do pipeline	job_id, decode_avg_ms, infer_avg_ms, p95, p99, effective_fps, peak_memory_mb

11. Camada de auditoria

A auditoria é o componente que transforma execução em conhecimento operacional. Seu propósito não é somente registrar logs, mas capturar o contexto completo da execução e permitir comparação entre cenários. A auditoria deve responder “o que rodou”, “como rodou”, “em que ambiente”, “com qual resultado” e “por que foi melhor ou pior que outra execução”.

11.1 Objetos de auditoria

Objeto	Função na auditoria
audit_run	Representa uma execução auditável e seus vínculos com job, perfil, engine, modelo e ambiente.
audit_parameter_snapshot	Congela todos os parâmetros efetivamente usados na execução.
audit_stage_metrics	Armazena métricas por estágio, incluindo médias, percentis, picos de memória e erros.
audit_comparison	Materializa o resultado comparativo entre baseline e candidato ou entre conjuntos de execuções.

11.2 O que deve ser congelado pela auditoria

- Assinatura do vídeo ou dataset usado na execução.
- Versão exata do modelo, do class mapping e da engine.
- Resolução, sampling, thresholds, batch size e tracking mode.
- Host, CPU, GPU, memória, sistema operacional e versão de driver.
- Política de fila, timeout, retry e pool de execução.
- Tempos por estágio, uso de memória, taxa de erro e resumo final do catálogo.

12. Algoritmização da auditoria comparativa

12.1 Regra de justiça experimental

Antes de comparar duas execuções, o sistema precisa validar se a comparação é justa. Se a comparação for de engines, modelo, vídeo, thresholds e perfil devem ser equivalentes. Se a comparação for de perfis, engine, modelo, vídeo e ambiente devem permanecer controlados. Toda divergência precisa ser registrada ou a comparação deve ser marcada como parcial.

Tipo de comparação	Variáveis que devem permanecer fixas	Variável-alvo
Perfis operacionais	Vídeo, modelo, engine, hardware, thresholds-base	Parâmetros do perfil
Engines de inferência	Vídeo, modelo equivalente, perfil, ambiente compatível	Backend de inferência
Versões de modelo	Vídeo, engine, perfil e ambiente	Modelo exportado
Ambientes	Vídeo, modelo, engine e perfil	Hardware / runtime environment

12.2 Algoritmo de execução auditável

```
executar_audit_run(job_config):
    audit_run = criar_audit_run(job_config)
    snapshot = congelar_parametros(job_config)
    validar_consistencia(snapshot)

    iniciar_coleta_metricas(audit_run)
    resultado = processar_job(job_config)
    finalizar_metricas(audit_run, resultado)

    persistir_deteccoes(resultado)
    persistir_catalogo(resultado)
    persistir_metricas_por_estagio(audit_run)
    calcular_indicadores_secundarios(audit_run)
    finalizar_audit_run(audit_run)
```

12.3 Algoritmo de comparação entre execuções

```
comparar_execucoes(execucoes, tipo_comparacao):
    validar_justica_experimental(execucoes, tipo_comparacao)
    normalizar_metricas(execucoes)

    para cada execucao em execucoes:
        calcular_indicadores_primarios(execucao)
        calcular_indicadores_secundarios(execucao)
        calcular_score_composto(execucao, tipo_comparacao)

    ordenar_por_score(execucoes)
    baseline = selecionar_baseline(execucoes)
    calcular_deltas_percentuais(baseline, execucoes)
    gerar_relatorio_comparativo(execucoes)
```

12.4 Indicadores primários

- Latência média total do job.
- Percentis p95 e p99 por estágio e para o job completo.
- FPS efetivo ou taxa equivalente de processamento.
- Throughput por worker e por pool.
- Uso médio e pico de CPU, GPU e memória.
- Taxa de erro, retries e abortos.

12.5 Indicadores secundários

- Estabilidade de latência: desvio padrão e amplitude entre execuções repetidas.
- Eficiência de memória: quantidade de frames ou objetos processados por unidade de memória.
- Robustez: combinação entre baixa taxa de erro e completude do processamento.
- Consistência de catálogo: proximidade entre catálogos gerados em repetições controladas.
- Eficiência operacional: objetos catalogados por unidade de tempo ou custo computacional.

12.6 Score composto

Para permitir ranking entre alternativas, o sistema pode calcular um score composto ajustado ao objetivo da comparação. Os pesos variam conforme a meta predominante do cenário auditado.

```
score_final =
    w_latencia      * score_latencia +
    w_throughput    * score_throughput +
    w_memoria       * score_memoria +
    w_estabilidade  * score_estabilidade +
    w_robustez      * score_robustez +
    w_consistencia  * score_consistencia
```

Cenário	Distribuição de pesos sugerida
Low Latency	Latência 0,35; Throughput 0,15; Memória 0,10; Estabilidade 0,15; Robustez 0,15; Consistência 0,10
High Throughput	Latência 0,15; Throughput 0,35; Memória 0,10; Estabilidade 0,10; Robustez 0,15;

Cenário	Distribuição de pesos sugerida
	Consistência 0,15
Cost Efficient	Latência 0,15; Throughput 0,15; Memória 0,25; Estabilidade 0,15; Robustez 0,15; Consistência 0,15
Balanced	Distribuição próxima do uniforme, com pequenos ajustes conforme o contexto operacional

A normalização dos indicadores deve distinguir métricas “quanto menor, melhor” e “quanto maior, melhor”. Sem normalização, qualquer score composto fica dominado por uma escala arbitrária.

```
# menor é melhor
score = (max_val - valor_atual) / (max_val - min_val)

# maior é melhor
score = (valor_atual - min_val) / (max_val - min_val)
```

13. Matriz de experimentos auditáveis

O sistema deve apoiar um catálogo de experimentos para que comparações não virem testes ad hoc. Cada experimento deve possuir objetivo, baseline, variáveis controladas, número mínimo de repetições e métricas obrigatórias.

Experimento	Objetivo	Controlados	Variável-alvo	Repetições mínimas
E01	Comparar perfis operacionais	Vídeo, modelo, engine, hardware	Parâmetros do perfil	10
E02	Comparar engines de inferência	Vídeo, modelo equivalente, perfil, hardware compatível	Engine	10
E03	Comparar versões de modelo	Vídeo, engine, perfil, hardware	Modelo	10
E04	Comparar ambientes	Vídeo, engine, perfil, modelo	Host / runtime	10

Repetições são importantes porque uma única execução raramente representa a distribuição real do comportamento operacional. A auditoria deve tratar média, dispersão e percentis como primeira classe.

14. Observabilidade integrada

14.1 Logs estruturados

- Todo log relevante deve carregar pelo menos job_id, audit_run_id, stage, worker_id e timestamp.

- Eventos de erro devem registrar código, mensagem, stack ou origem funcional, além de dados suficientes para replay.
- O padrão de log deve ser consistente entre ingestão, dispatcher e worker, permitindo correlação ponta a ponta.

14.2 Métricas operacionais

- Profundidade da fila por pool e tipo de job.
- Jobs recebidos, ativos, concluídos, falhos e reenfileirados.
- Tempo médio e percentis por estágio.
- Uso de memória e CPU/GPU por worker.
- FPS efetivo por perfil e engine.

14.3 Tracing lógico

```
ingestion_request_id -> job_id -> audit_run_id -> worker_id -> export_id
```

15. Estados do job e tolerância a falhas

Estado	Significado operacional
RECEIVED	Submissão registrada, ainda não encaminhada para execução.
QUEUED	Job aguardando disponibilidade do pool correspondente.
RUNNING	Job em processamento ativo por um worker.
PARTIAL	Job produziu parte dos artefatos, mas encerrou com degradação.
RETRYING	Nova tentativa disparada por política de tolerância a falhas.
FAILED	Processamento interrompido sem sucesso.
COMPLETED	Processamento concluído com persistência dos artefatos previstos.
CANCELLED	Execução interrompida por decisão externa ou regra administrativa.

- Vídeo corrompido deve ser distinguido de erro interno de worker.
- Falha de storage não deve apagar o estado do job nem a trilha parcial já coletada.
- Timeout precisa ser registrado com contexto do estágio em execução.
- Dead-letter queue é recomendada para jobs que excedam o número máximo de retries.

16. Segurança, governança e integridade operacional

- Separar credenciais de storage, broker e banco de dados por ambiente.
- Versionar modelos, configurações e mapeamentos de classes para evitar ambiguidade de resultado.
- Assinar configuração efetiva da execução por hash e armazenar essa assinatura junto ao audit_run.

- Aplicar política de retenção para vídeos, frames amostrados, exportações e logs, distinguindo dados quentes e frios.
- Prever anonimização ou mascaramento caso o pipeline seja aplicado a vídeos com dados sensíveis.

17. Roadmap de implementação

Fase	Objetivo central	Entregas detalhadas
Fase 1	MVP técnico fim a fim	Leitura de vídeo, inferência em C, pós-processamento básico, exportação simples e persistência mínima.
Fase 2	Worker modular	Separação de decoder, preprocess, backend, postprocess, tracker, catálogo e métricas por estágio.
Fase 3	Arquitetura distribuída	Serviço de ingestão, broker, dispatcher, múltiplos workers e persistência operacional completa.
Fase 4	Auditoria comparativa	audit_run, snapshots, comparação entre execuções, score composto e relatórios auditáveis.
Fase 5	Otimização e robustez	Retry, dead-letter, tuning de perfis, benchmark contínuo, melhora de tracking e autoscaling.

18. Backlog estruturado por temas

18.1 Tema A — Processamento

- Épico A1: decoder e sampler confiáveis.
- Épico A2: engine de inferência plugável.
- Épico A3: pós-processamento e tracking básico.
- Épico A4: catalogação e exportação.

18.2 Tema B — Plataforma

- Épico B1: ingestão e cadastro de jobs.
- Épico B2: fila, scheduler e políticas de despacho.
- Épico B3: persistência operacional e consulta.
- Épico B4: observabilidade e tolerância a falhas.

18.3 Tema C — Auditoria

- Épico C1: trilha auditável completa.
- Épico C2: coleta de métricas primárias e secundárias.
- Épico C3: matriz de experimentos comparáveis.
- Épico C4: score composto e ranking por cenário.

19. Principais riscos e mitigação

Risco	Efeito potencial	Mitigação recomendada
Acoplamento à engine	Dificulta benchmark e substituição	Manter contrato uniforme de inferência e testes por backend.
Comparação injusta	Conclusões técnicas inválidas	Validar justiça experimental antes de comparar.
Gargalo de decodificação	Mascaramento do desempenho da CNN	Métricas por estágio e profiling dedicado do decoder.
Catálogo inflado	Resumo enganoso de objetos	Tracking básico e contagem por track quando aplicável.
Falta de snapshot completo	Auditoria irreplicável	Congelar parâmetros, ambiente e hashes de configuração.

20. Critérios de sucesso

11. O pipeline em C processa vídeos fim a fim com detecção e catalogação consistentes.
12. A arquitetura suporta distribuição por múltiplos workers.
13. Cada execução produz métricas por estágio e trilha auditável suficiente para replay analítico.
14. A engine de inferência pode ser trocada sem alterar o pipeline de domínio.
15. Perfis operacionais podem ser comparados de forma controlada.
16. A organização consegue responder, com dados, qual engine e qual perfil entregam melhor resultado para cada cenário.

21. Recomendação de implementação prática

A sequência de construção mais segura é: primeiro consolidar o pipeline único em C, depois encapsular a engine, em seguida introduzir persistência e auditoria local, depois transformar o pipeline em worker modular e, por fim, externalizar ingestão, fila e scheduler. Essa ordem preserva foco técnico, reduz risco de arquitetura vazia e garante que cada camada nova seja apoiada por um core realmente funcional.

17. Construir e medir o pipeline único.
18. Criar a abstração da inferência e provar pelo menos um backend funcional.
19. Persistir detecções, catálogo e métricas locais.
20. Formalizar o objeto audit_run e o snapshot de parâmetros.
21. Introduzir o worker modular e, depois, a arquitetura distribuída.
22. Institucionalizar benchmark contínuo e relatórios de comparação.

Apêndice A — Exemplo de layout de código

```
vision_platform/
```

```
├── cmd/
│   ├── ingestion_service/
│   ├── worker/
│   ├── scheduler/
│   └── export_service/
├── core/
│   ├── domain/
│   ├── pipeline/
│   ├── infra/
│   └── common/
├── deployments/
├── models/
├── configs/
├── scripts/
└── docs/
```

Apêndice B — Exemplo de estrutura de comparação

```
comparacao_engine(
    video = V001,
    perfil = BALANCED,
    modelo = YOLO_ONNX_V1,
    engines = [ONNX_RUNTIME, TENSORRT, OPENVINO],
    repeticoes = 10,
    metricas = [latencia_media, p95, throughput, pico_memoria, taxa_erro,
consistencia_catalogo]
)
```